

2026 春

《程序设计实习》

程序设计实习 第八讲

标准模板库1

杨 帅

williamyang@pku.edu.cn

ITERATORS
(Connectors)

ALGORITHMS
(Data Processing)

ALGORITHMS
(Data Processing)

CONTAINERS
(Data Structures)

`std::vector<int>`

`std::list<double>`

`std::vector<double>`

`std::map<string, int>`

`v.begin()`
`v.begin()`

`#include <algorithm>`
`std::sort(v.begin(), v.end());`

`std::sort(v.begin(), v.end());`
`std::sort(v.begin(), v.end());`

课前多叽歪

- **本周清明假期上机停一次**→ 期中前还有3次上机
- 4月课程安排：C++高阶学习
 - [4.03] 08. 标准模板库STL-1
 - [4.08] 09. 标准模板库STL-2
 - [4.15] 10. 标准模板库STL-3和C++高阶
 - [4.17] 11. C++新特性
 - [4.22] 12. QT技术分享
 - [4.29] **期中考试**

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

- **STL基本概念**
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

- C++语言的核心优势之一 —— **软件的重用**
- C++中有两个方面体现重用：
 1. **面向对象**的思想：继承和多态，标准类库
 2. **泛型程序设计**（generic programming）的思想：模板机制，以及标准模板库 STL

■ 就是使用**模板**的程序设计法

- 将一些常用的**数据结构**(比如链表, 数组, 二叉树)和**算法**(比如排序, 查找)写成模板
- 以后不论数据结构里放的是什麼对象, 算法针对什麼样的对象
→ 都不必重新实现数据结构, 重新编写算法

■ 标准模板库 (Standard Template Library)

- 常用**数据结构和算法的模板的集合**
- 不必再写太多的标准数据结构和算法
- 并且可获得非常高的性能

STL中的基本的概念

C++ STL
Standard Template Library
Ready-to-use Components

《程序设计实习》

- **容 器**：可容纳各种数据类型的通用数据结构，是类模板
- **迭代器**：可用于依次存取容器中元素，类似于指针
 - 普通的C++指针就是一种迭代器
- **算 法**：用来操作容器中的元素的函数模板
 - `sort()`来对一个数组中的数据进行排序
 - `find()`来搜索一个数组中的对象

算法本身与他们操作的数据的类型无关，

因此他们可以在从简单数组到高度复杂容器的任何数据结构上使用

```
int array[100];           // 该数组就是容器，而int* 类型的指针变量就可以作为迭代器
sort(array, array+70);    // sort 算法可以作用于该容器上，对其进行排序
```

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

- 可以用于存放各种类型的数据（基本类型的变量，对象等）的数据结构，都是类模版，分为三种：

1. 顺序容器/序列容器

`vector`, `deque`, `list`

2. 关联容器/有序容器

`set`, `multiset`, `map`, `multimap`

3. 容器适配器

`stack`, `queue`, `priority_queue`

第一类容器

- 对象被插入容器中时，被插入的是**对象的一个复制品**
- 许多算法，比如排序，查找，要求对容器中的元素进行比较，有的容器本身就是排序的
- 所以放入容器的**对象所属的类**，往往还应该**重载 == 和 < 运算符**

顺序容器简介

《程序设计实习》

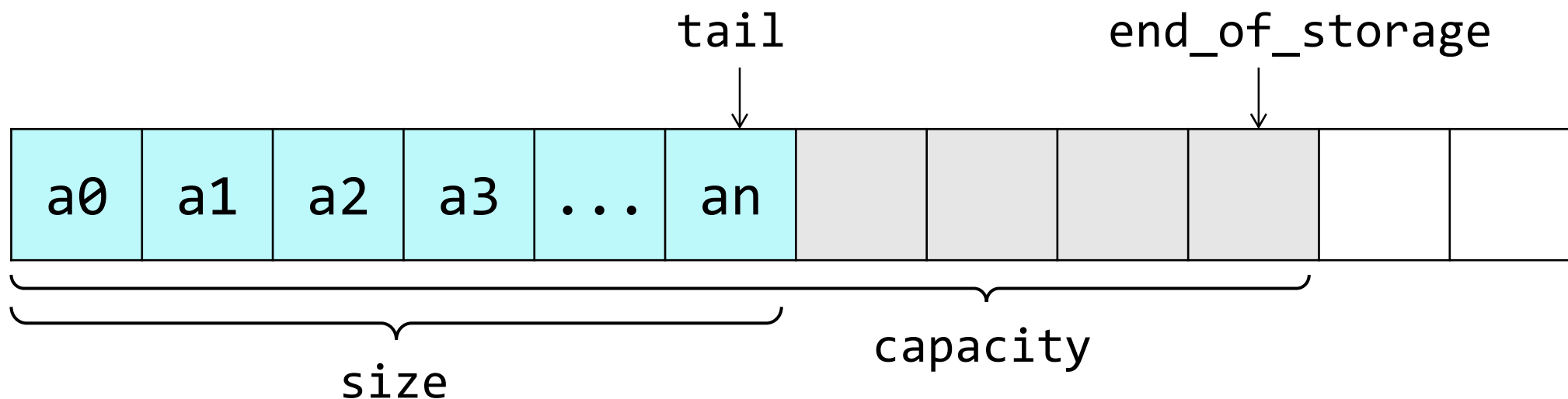
- 容器并非排序的，元素的插入位置同元素的值无关
- 有vector，deque，list三种

■ vector 头文件 <vector>

动态数组，

随机存取任何元素都能在常数时间完成，

在尾端增删元素具有较佳的性能

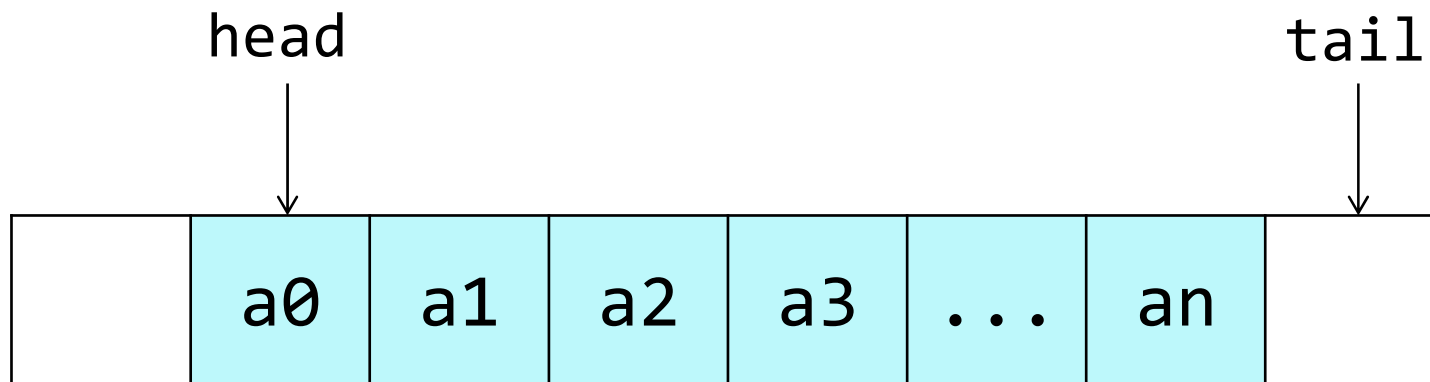


■ deque 头文件 <deque>

双向队列，元素在内存连续存放

随机存取任何元素都能在常数时间完成(但次于vector)

在两端增删元素具有较佳的性能(大部分情况下是常数时间)

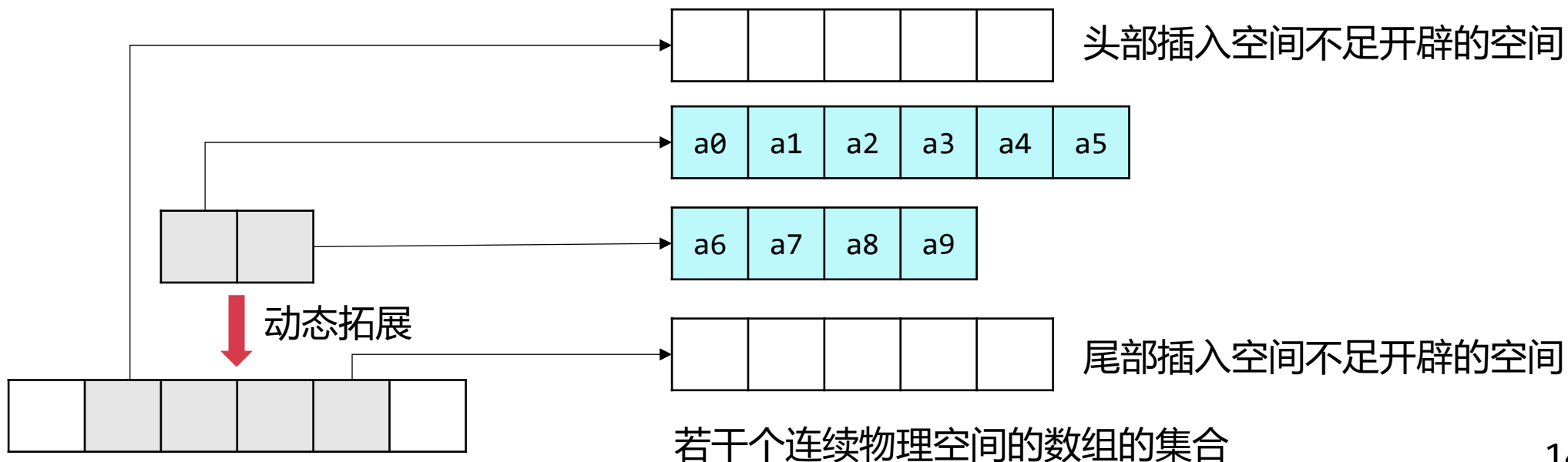


■ deque 头文件 <deque>

双向队列，元素在内存连续存放

随机存取任何元素都能在常数时间完成(但次于vector)

在两端增删元素具有较佳的性能(大部分情况下是常数时间)

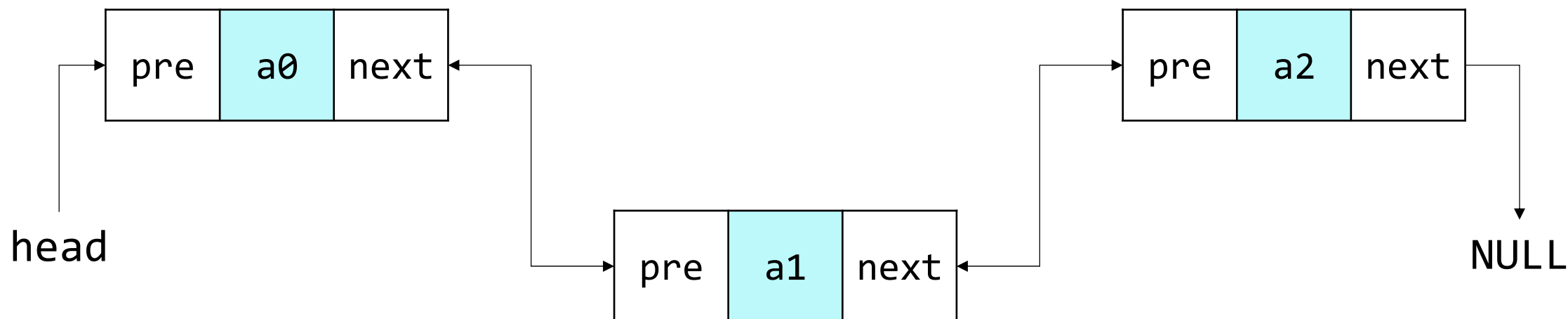


■ list 头文件 <list>

双向链表，元素在内存不连续存放

在任何位置增删元素都能在常数时间完成

不支持随机存取



■ 元素是**排序**的

- 插入任何元素，都按相应的排序规则来确定其位置
- 在查找时具有非常好的性能
- 通常以平衡二叉树方式实现，**插入和检索的时间都是 $O(\log N)$**

■ **set/multiset** 头文件 **<set>**

集合，set不允许相同元素，multiset中允许存在相同的元素

■ **map/multimap** 头文件 **<map>**

映射，map与set的不同在于map中存放的是**成对的key/value**

并根据key对元素进行排序，可快速地根据key来检索元素

map同multimap的不同在于是否允许相同key的元素

■ 适配器的含义

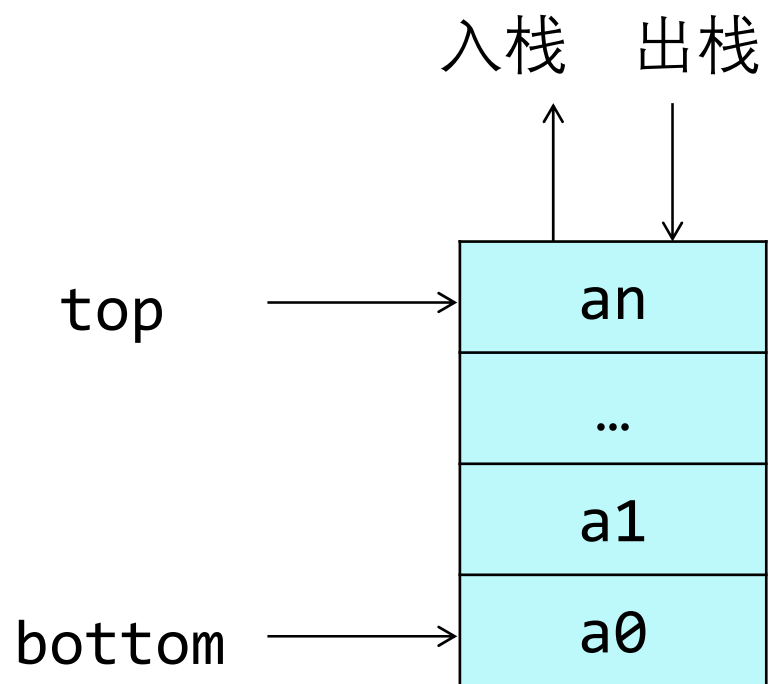
- 容器适配器，就是将不适用的顺序容器(包括vector、deque 和list)变得适用
- 即通过封装某个顺序容器，并重新组合该容器中包含的成员函数，使其满足某些特定场景的需要
- 容器适配器本质上还是容器，只不过此容器模板类的实现，利用了大量其它基础容器模板类中已经写好的成员函数

■ `stack` 头文件 `<stack>`

栈.

项的有限序列，并满足序列中被删除、检索和修改的项只能是最近插入序列的项(栈顶的项)

即按照**后进先出**的原则

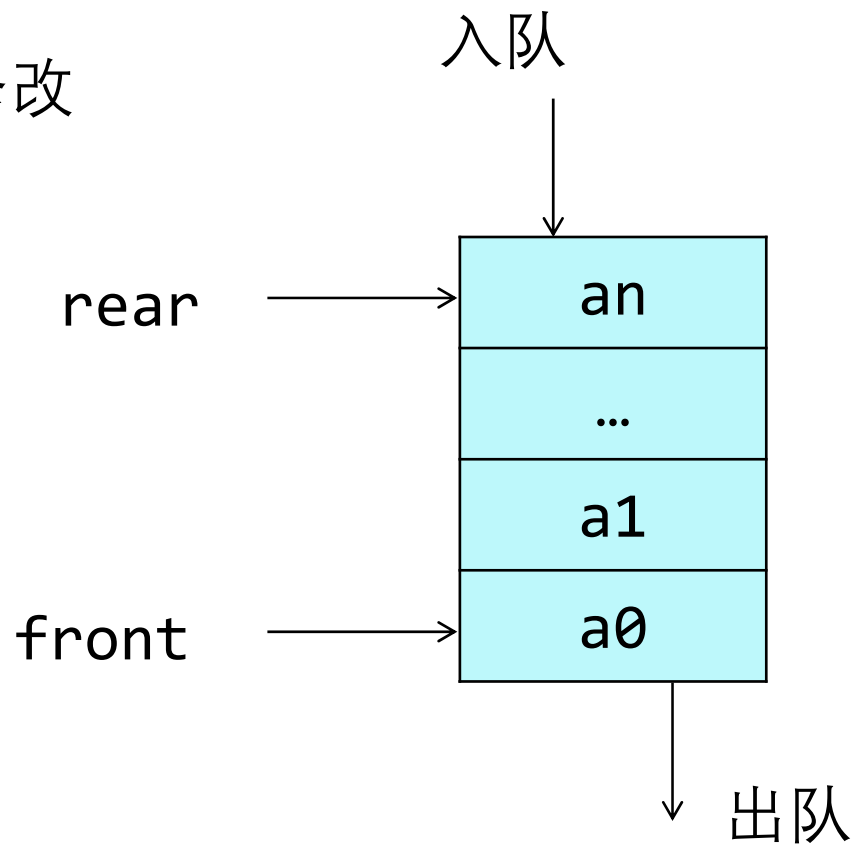


■ queue 头文件 <queue>

队列

插入只可以在尾部进行，删除、检索和修改
只允许从头部进行

按照**先进先出**的原则



■ `priority_queue` 头文件 `<queue>`

优先级队列

最高优先级元素总是第一个出列

■ 所有标准库容器共有的成员函数

- 相当于按词典顺序比较两个容器的运算符：

=, <, <=, >, >=, ==, !=

- `empty`: 判断容器中是否有元素
- `max_size`: 容器中最多能装多少元素
- `size`: 容器中元素个数
- `swap`: 交换两个容器的内容

■ 比较两个容器的例子

```
#include <vector>
#include <iostream>
using namespace std;
class A {
private:
    int n;
public:
    friend bool operator < (const A &, const A &);
    A( int n_ ) { n = n_ ; }
};
bool operator < (const A & o1, const A & o2) {
    return o1.n < o2.n;
}
```

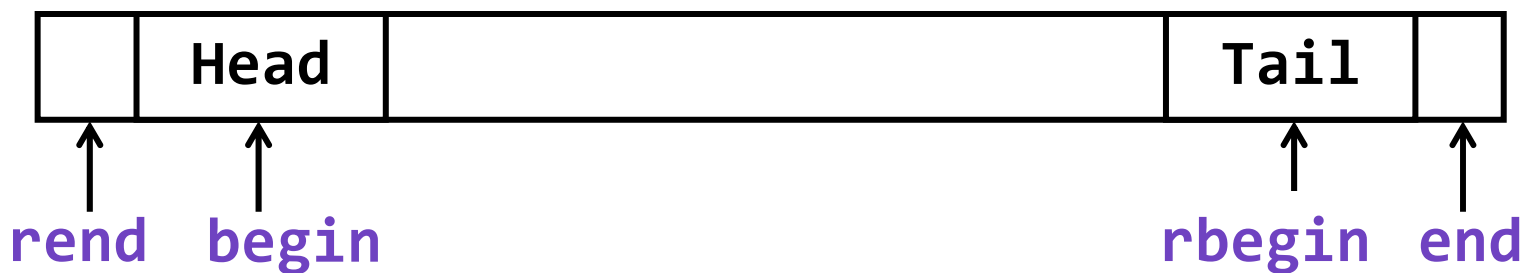
```
int main(){
    vector<A> v1;
    vector<A> v2;
    v1.push_back(A(5));
    v1.push_back(A(1));
    v2.push_back(A(1));
    v2.push_back(A(2));
    v2.push_back(A(3));
    cout << (v1 < v2);
    return 0;
}
```

0

只在第一类容器中的成员函数

《程序设计实习》

- **begin** 返回指向容器中**第一个元素**的迭代器
- **end** 返回指向容器中**最后一个元素后面**的位置的迭代器
- **rbegin** 返回指向容器中**最后一个元素**的迭代器
- **rend** 返回指向容器中**第一个元素前面**的位置的迭代器
- **erase** 从容器中删除一个或几个元素
- **clear** 从容器中删除所有元素



顺序容器的常用成员函数

《程序设计实习》

- **front** 返回容器中**第一个元素**的引用
- **back** 返回容器中**最后一个元素**的引用
- **push_back** 在容器末尾增加新元素
- **pop_back** 删除容器末尾的元素
- **erase** 删除迭代器指向的元素(可能会使该迭代器失效),
或删除一个区间, 返回被删除元素后面的那个元素的迭代器

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

- 用于指向**第一类容器**中的元素，有const和非const两种
 - 通过迭代器可以读取它指向的元素
- 通过非const迭代器还能修改其指向的元素
- 迭代器用法和指针类似

//定义一个容器类的迭代器的方法:

容器类名::iterator 变量名;

容器类名::const_iterator 变量名;

//访问一个迭代器指向的元素:

* 迭代器变量名

- 迭代器上可以**执行 ++** 操作，以使其指向容器中的下一个元素
- 如果迭代器到达了容器中的最后一个元素的后面
 - 迭代器变成 `past-the-end` 值
 - 此时再使用它，就会出错
 - 类似于使用 `NULL` 或未初始化的指针一样

迭代器示例

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
    vector<int> v; //一个存放int元素的数组, 一开始里面没有元素
    v.push_back(1); v.push_back(2); v.push_back(3); v.push_back(4);
    vector<int>::const_iterator i; //常量迭代器
    for( i = v.begin(); i != v.end(); i++ )
        cout << * i << ", ";
    cout << endl;
    vector<int>::reverse_iterator r; //反向迭代器
    for( r = v.rbegin(); r != v.rend(); r++ )
        cout << * r << ", ";
    cout << endl;
    vector<int>::iterator j; //非常量迭代器
    for( j = v.begin(); j != v.end(); j++ )
        * j = 100;
    for( i = v.begin(); i != v.end(); i++ )
        cout << * i << ", ";
    return 0;
}
```

```
1,2,3,4,
4,3,2,1,
100,100,100,100,
```

- 不同容器上支持的迭代器功能强弱有所不同
- 功能强弱，决定了该容器**是否支持STL中的某种算法**
 - 只有**第一类容器**能用迭代器遍历
 - 排序算法需要通过随机迭代器来访问容器中的元素
 - 那么有的容器就不支持排序算法

- STL中的迭代器按功能由弱到强分为5种
 1. 输入：提供对数据的只读访问
 1. 输出：提供对数据的只写访问
 2. 正向：提供读写操作，并能一次一个地向前进迭代器
 3. 双向：提供读写操作，并能一次一个地向前和向后移动
 4. 随机访问：提供读写操作，并能在数据中随机移动
- 编号大的迭代器拥有编号小的迭代器的所有功能，能当作编号小的迭代器使用

不同迭代器所能进行的操作(功能)

《程序设计实习》

- 所有迭代器: `++p`, `p++`
- 输入迭代器: `value = * p`, `p = p1`, `p == p1`, `p != p1`
- 输出迭代器: `* p = value`, `p = p1`
- 正向迭代器: 上面全部
- 双向迭代器: 上面全部, `--p`, `p--`,
- 随机访问迭代器: 上面全部, 以及:
 - **移动*i*个单元**: `p += i`, `p -= i`, `p + i`, `p - i`
 - **大于/小于比较**: `p < p1`, `p <= p1`, `p > p1`, `p >= p1`
 - **数组下标*p[i]***: `p` 后面的第*i*个元素的引用

容器	迭代器类别
vector	随机
deque	随机
list	双向
set/multiset	双向
map/multimap	双向
stack	不支持迭代器
queue	不支持迭代器
priority_queue	不支持迭代器

容器	迭代器类别
vector	随机
deque	随机
list	双向
set/multiset	双向
map/multimap	双向
stack	不支持迭代器
queue	不支持迭代器
priority_queue	不支持迭代器

有的算法，例如sort，binary_search需要通过**随机访问迭代器**来访问容器中的元素，那么list以及关联容器就不支持该算法！

- vector的迭代器是随机迭代器，遍历方法如下(deque亦然):

```
vector<int> v(100);
int i;
for(i = 0; i < v.size(); i ++ )
    cout << v[i];
vector<int>::const_iterator ii;
for( ii = v.begin(); ii != v.end(); ii ++ )
    cout << * ii;
for( ii = v.begin(); ii < v.end(); ii ++ )
    cout << * ii;
    ii = v.begin();
while( ii < v.end() ) { //间隔一个输出
    cout << * ii;
    ii = ii + 2;
}
```

- list的迭代器是双向迭代器，正确的遍历方法如下：

```
list<int> v;  
list<int>::const_iterator ii;  
for( ii = v.begin(); ii != v.end(); ii ++ )  
    cout << * ii;
```

- 错误的方法：

```
for( ii = v.begin(); ii < v.end(); ii ++ ) //双向迭代器不支持 <  
    cout << * ii;  
for(int i = 0; i < v.size(); i ++)  
    cout << v[i];    //双向迭代器不支持 []
```

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

■ STL提供能在各种容器中通用的算法

- 算法就是一个个**函数模板**，大多数在<algorithm>中定义
- 算法通过迭代器来操纵容器中的元素。
- 许多算法可以对容器中的一个局部区间进行操作，因此需要两个参数，一个是**起始元素的迭代器**，一个是**终止元素的后面一个元素的迭代器**
 - 排序和查找
- 有的算法**返回一个迭代器**。比如 find() 算法，在容器中查找一个元素，并返回一个指向该元素的迭代器
- 算法可以处理**容器**，也可以处理**C语言普通数组**

■ STL提供能在各种容器中通用的算法

copy, remove,
fill, replace,
random_shuffle,
swap, ...

变化序列算法

equal, find,
count, search,
count_if,
for_each, ...

非变化序列算法

- 以上函数模板都在 `<algorithm>` 中定义
- 还有其他算法, 例如 `<numeric>` 中的算法

算法示例：find()

C++ STL

(Standard Template Library)
Ready-to-Use Components

《程序设计实习》

```
template<class InIt, class T>
InIt find(InIt first, InIt last, const T& val);
```

■ 查找区间

- first 和 last 都是容器的迭代器，它们给出了容器中的查找区间起点和终点[first, last)
- 这个区间是个左闭右开的区间，即区间的起点是位于查找范围之中的，而终点不是

■ 查找对象

- find在[first, last)查找等于val的元素，用 == 运算符判断相等

■ 返回值是一个迭代器

- 如果找到，则该迭代器**指向被找到的元素**
- 如果找不到，则该迭代器**等于last**

算法示例：find()

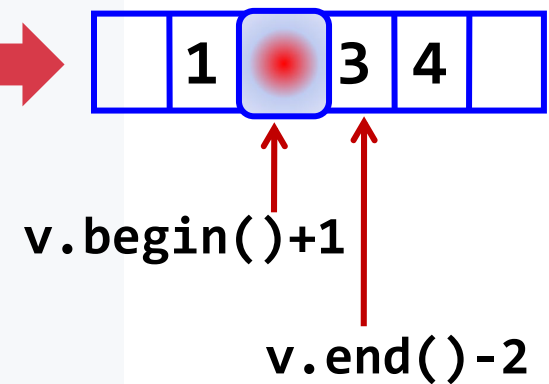
C++ STL

(Standard Template Library)
Ready-to-Use Components

《程序设计实习》

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
int main() {
    vector<int> v;
    v.push_back(1); v.push_back(2); v.push_back(3); v.push_back(4);
    vector<int>::iterator p;
    p = find(v.begin(), v.end(), 3);
    if( p != v.end() ) cout << * p << endl;
    p = find(v.begin(), v.end(), 9);
    if( p == v.end() ) cout << "not found " << endl;
    p = find(v.begin()+1, v.end()-2, 1); //查找区间[2, 3)
    if( p != v.end() ) cout << * p << endl;
    int array[10] = {10, 20, 30, 40};
    int * pp = find(array, array+4, 20);
    cout << * pp << endl;
    return 0;
}
```

3
not found
3
20



- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

■ 比大小

- 关联容器内部的元素是从小到大排序的
- 有些算法(binary_search)要求其操作的区间是从小到大排序的，称为"有序区间算法"
- 有些算法(sort)会对区间进行从小到大排序，称为"排序算法"

■ < 运算符

- STL中，缺省的情况下，比较大小是用 "<" 运算符进行的，和 ">" 运算符无关
- 使用STL时，在缺省的情况下，以下三个说法等价：
 - x 比 y 小
 - 表达式 "x < y" 为真
 - y 比 x 大

■ "x和y相等" vs. "x==y为真"

- 有时, "x和y相等" 等价于 "x==y为真"

- 在**未排序**的区间上进行的算法
- 例如: 顺序查找find

- 有时, "x和y相等" 等价于 "**x<y和y<x同时为假**"

- 对于在**排好序**的区间上进行查找, 合并等操作的算法
- 与 == 运算符无关
- 例如: 折半查找算法binary_search, 关联容器自身的成员函数find

STL中"相等"概念演示

《程序设计实习》

```
#include <iostream>
#include <algorithm>
using namespace std;
class A {
    int v;
public:
    A(int n):v(n) { }
    bool operator < (const A & a2) const {
        cout << v << "<" << a2.v << "?" << endl;
        return false;
    }
    bool operator == (const A & a2) const {
        cout << v << "==" << a2.v << "?" << endl;
        return v == a2.v;
    }
};
int main(){
    A a[] = { A(1), A(2), A(3), A(4), A(5) };
    cout << binary_search(a, a+4, A(9)); //折半查找
    return 0;
}
```

3<9?
2<9?
1<9?
9<1?
1

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器： **vector**和双向队列**deque**
- 顺序容器： 双向链表**list**
- 函数对象

顺序容器的常用成员函数

《程序设计实习》

- **front** 返回容器中**第一个元素**的引用
- **back** 返回容器中**最后一个元素**的引用
- **push_back** 在容器**末尾增加**新元素
- **pop_back** **删除**容器**末尾**的元素

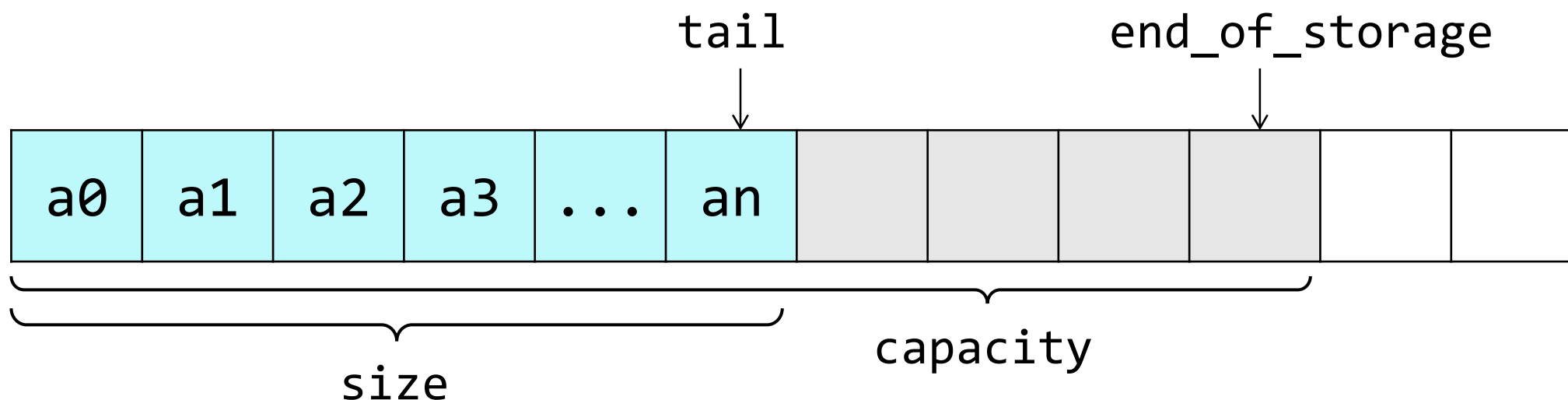
■ vector 头文件 <vector>

动态数组

支持随机访问迭代器，所有STL算法都能对vector操作

随机访问时间为常数

在尾端增删元素具有较佳的性能，在中间插入慢



```
int main() {  
    int i;  
    int a[5] = { 1, 2, 3, 4, 5 };  
    vector<int> v(5);  
    cout << v.end() - v.begin() << endl;  
    for( i = 0; i < v.size(); i ++ )  
        v[i] = i;  
    v.at(4) = 100;  
    for( i = 0; i < v.size(); i ++ )  
        cout << v[i] << ", " ;  
    cout << endl;  
    vector<int> v2(a, a+5); //构造函数  
    v2.insert( v2.begin() + 2, 13 ); //在begin()+2位置插入 13  
    for( i = 0; i < v2.size(); i ++ )  
        cout << v2[i] << ", " ;  
    return 0;  
}
```

```
5  
0, 1, 2, 3, 100,  
1, 2, 13, 3, 4, 5,
```

用vector实现二维数组

《程序设计实习》

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<vector<int>> v(3);
    //v有3个元素，每个元素都是vector<int>容器
    for(int i = 0; i < v.size(); ++i)
        for(int j = 0; j < 4; ++j)
            v[i].push_back(j);
    for(int i = 0; i < v.size(); ++i) {
        for(int j = 0; j < v[i].size(); ++j)
            cout << v[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

```
0 1 2 3
0 1 2 3
0 1 2 3
```

vector示例程序

```
1) 5
2) 1 2 3 4 5
3) 1 2 13 3 4 5
4) 1 2 3 4 5
5) v2: 2 3 100 100 100 100
6) 1 4 5
```

```
template<class T>
void PrintVector(T s, T e) {
    for(; s != e; ++s)
        cout << *s << " ";
    cout << endl;
}

int main() {
    int a[5] = { 1,2,3,4,5 };
    vector<int> v(a,a+5);           //将数组a的内容放入v
    cout << "1) " << v.end() - v.begin() << endl;
    cout << "2) "; PrintVector(v.begin(), v.end());
    v.insert(v.begin() + 2, 13);    //在begin()+2位置插入 13
    cout << "3) "; PrintVector(v.begin(), v.end());
    v.erase(v.begin() + 2);         //删除位于 begin() + 2的元素
    cout << "4) "; PrintVector(v.begin(), v.end());
    vector<int> v2(4,100);           //v2 有4个元素, 都是100
    v2.insert(v2.begin(), v.begin() + 1, v.begin() + 3);
    cout << "5) v2: "; PrintVector(v2.begin(), v2.end());
    v.erase(v.begin() + 1, v.begin() + 3);
    cout << "6) "; PrintVector(v.begin(), v.end());
    return 0;
}
```


vector示例程序

■ 花样输出

```
1) 5
2) 1 2 3 4 5
3) 1 2 13 3 4 5
4) 1 2 3 4 5
5) v2: 2 3 100 100 100 100
6) 1 4 5
```

```
#include<iostream>
#include<vector>
#include<iterator>
using namespace std;

int main() {
    ostream_iterator<int> output(cout, " ");
    int a[5] = { 1,2,3,4,5 };
    vector<int> v(a,a+5);           //将数组a的内容放入v
    cout << "1) " << v.end() - v.begin() << endl;
    cout << "2) "; copy (v.begin(), v.end(), output);
    v.insert(v.begin() + 2, 13);    //在begin()+2位置插入 13
    cout << "\n3) "; copy (v.begin(), v.end(), output);
    v.erase(v.begin() + 2);         //删除位于 begin() + 2的元素
    cout << "\n4) "; copy (v.begin(), v.end(), output);
    vector<int> v2(4,100);           //v2 有4个元素, 都是100
    v2.insert(v2.begin(), v.begin() + 1, v.begin() + 3);
    cout << "\n5) v2: "; copy (v2.begin(), v2.end(), output);
    v.erase(v.begin() + 1, v.begin() + 3);
    cout << "\n6) "; copy (v.begin(), v.end(), output);
    return 0;
}
```

■ 函数模板copy:

- STL算法，用于将一个范围内的元素复制到另一个范围
- `copy (v.begin(), v.end(), output);`
- 导致v的内容在通过 output 上输出，为什么？

```
// copy类似于
template <class T1, class T2>
T2 Copy(T1 s, T1 e, T2 x){
    for(; s != e; ++s, ++x)
        *x = *s;
    return x;
}
```

- 模板类 **ostream_iterator**: 写入输出流
 - `ostream_iterator<int> output(cout, " ");`
 - 定义了一个输出迭代器对象 `output`,
 - 它将数据写入到 `cout` 中, 在每次写入数据后插入的分隔符 `" "`
- 模板类 **istream_iterator**: 读取输入流

ostream_iterator

C++ STL

(Standard Template Library)
Ready-to-Use Components

《程序设计实习》

```
#include<iostream>
#include <iterator>
using namespace std;

int main() {
    istream_iterator<int> inputInt(cin);
    int n1, n2;
    n1 = * inputInt;    //读入 n1
    inputInt ++;
    n2 = * inputInt;    //读入 n2
    cout << n1 << ", " << n2 << endl;
    ostream_iterator<int> outputInt(cout);
    * outputInt = n1 + n2;
    cout << endl;
    int a[5] = {1, 2, 3, 4, 5};
    copy(a, a+5, outputInt);    //输出整个数组
    return 0;
}
```

键盘输入:

1 2 ↵

输出:

1, 2

3

12345

思考

■ 上页的ostream_iterator类应该重载以下运算符中的几个? =, *, ++, !=

A 1

B 2

C 3

D 4

思考

■ 上页的ostream_iterator类应该重载以下运算符中的几个? =, *, ++, !=

A 1

B 2

C 3

D 4

```
// copy类似于
template <class T1, class T2>
T2 Copy(T1 s, T1 e, T2 x){
    for(; s != e; ++s, ++x)
        *x = *s;
    return x;
}
```


思考

■ 上页的ostream_iterator类将输出的工作放在哪个运算符中做最合适?

A ++

B =

C *

D !=

思考

■ 上页的ostream_iterator类将输出的工作放在哪个运算符中做最合适？

A ++

B =

C *

D !=

```
// copy类似于
template <class T1, class T2>
T2 Copy(T1 s, T1 e, T2 x){
    for(; s != e; ++s, ++x)
        *x = *s;
    return x;
}
```

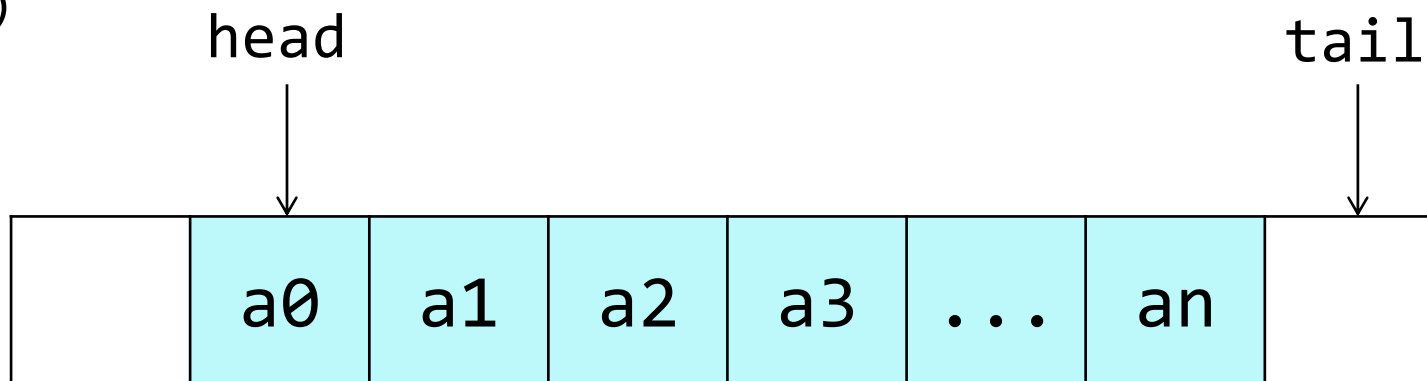
模仿ostream_iterator

《程序设计实习》

```
template<class T>
class My_ostream_iterator{
private:
    string sep;        //分隔符
    ostream & os;
public:
    My_ostream_iterator(ostream & o, string s):sep(s), os(o){ }
    void operator ++() { }        // ++只需要有定义即可
    My_ostream_iterator & operator * ()
    {    return * this;    }
    My_ostream_iterator & operator = ( const T & val )
    {    os << val << sep;    return * this;    }
};
```

■ deque容器

- 所有适用于vector的操作都适用于deque
- 比vector的优点：头部删除/添加元素性能也很好
- deque还包含以下操作：
 - **push_front**：将元素插入到前面
 - **pop_front**：删除最前面的元素
 - 复杂度都是 $O(1)$



- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

■ list 头文件 <list>

双向链表，在任何位置增删元素都是**常数时间**，**不支持随机存取**

除了具有所有顺序容器都有的成员函数以外，还支持8个成员函数

- **push_front**: 在**前面插入**
- **pop_front**: **删除前面**的元素
- **sort**: 排序 (**list**不支持STL的算法**sort**)
- **remove**: 删除和指定值相等的所有元素
- **unique**: 删除所有和前一个元素相同的元素 (要做到无重复元素，还需先**sort**)
- **merge**: 合并两个链表，并清空被合并的那个
- **reverse**: 颠倒链表
- **splice**: 在指定位置前面插入另一链表中的一个或多个元素，并在另一链表中删除被插入的元素

list容器之sort函数

《程序设计实习》

- list容器的迭代器不支持完全随机访问，所以不能用标准库中sort函数对它进行排序
- list自己的sort成员函数

```
list<int> classname;  
classname.sort(compare);    //compare函数可以自己定义  
classname.sort();           //无参数版本，按<排序
```

- 与其他顺序容器不同，list容器只能使用双向迭代
 - 不支持大于/小于比较运算符，[]运算符和随机移动
(即类似 "list的迭代器+2"的操作)

list示例程序

```
#include <list>
#include <iostream>
#include <algorithm>
using namespace std;
class A {    //定义类A, 并以友元重载<, ==和<<
private:
    int n;
public:
    A( int n_ ) { n = n_; }
    friend bool operator<( const A & a1,  const A & a2 );
    friend bool operator==( const A & a1,  const A & a2 );
    friend ostream & operator <<( ostream & o,  const A & a );
};
bool operator<( const A & a1,  const A & a2 ) {
    return a1.n < a2.n;
}
bool operator==( const A & a1,  const A & a2 ) {
    return a1.n == a2.n;
}
ostream & operator <<( ostream & o,  const A & a ) {
    o << a.n;
    return o;
}
```

list示例程序

《程序设计实习》

//定义函数模板PrintList，打印列表中的对象

```
template <class T>
void PrintList(const list<T> & lst) {
    int tmp = lst.size();
    if( tmp > 0 ) {
        typename list<T>::const_iterator i;
        for( i = lst.begin(); i != lst.end(); i ++ )
            cout << * i << ", ";
    }
}
```

// typename用来说明 list<T>::const_iterator是个类型，在VS中不写也可以

```
int main() {
    list<A> lst1, lst2;
    lst1.push_back(1); lst1.push_back(3);
    lst1.push_back(2); lst1.push_back(4);
    lst1.push_back(2); lst2.push_back(10);
    lst2.push_front(20); lst2.push_back(30);
    lst2.push_back(30); lst2.push_back(30);
    lst2.push_front(40); lst2.push_back(40);
    cout << "1) "; PrintList(lst1); cout << endl;
    cout << "2) "; PrintList(lst2); cout << endl;
}
```

1) 1, 3, 2, 4, 2,
2) 40, 20, 10, 30, 30, 30, 40,

list示例程序

《程序设计实习》

```
lst2.sort();          //list容器的sort函数
cout << "3) "; PrintList(lst2); cout << endl;
lst2.pop_front();
cout << "4) "; PrintList(lst2); cout << endl;
lst1.remove(2);       //删除所有和A(2)相等的元素
cout << "5) "; PrintList(lst1); cout << endl;
lst2.unique();        //删除所有和前一个元素相等的元素
cout << "6) "; PrintList(lst2); cout << endl;
lst1.merge(lst2);     //合并 lst2到lst1并清空lst2
cout << "7) "; PrintList(lst1); cout << endl;
cout << "8) "; PrintList(lst2); cout << endl;
lst1.reverse();
cout << "9) "; PrintList(lst1); cout << endl;
```

```
lst1 1, 3, 2, 4, 2,
lst2 40, 20, 10, 30, 30, 30, 40,

3) 10, 20, 30, 30, 30, 40, 40,

4) 20, 30, 30, 30, 40, 40,

5) 1, 3, 4,

6) 20, 30, 40,

7) 1, 3, 4, 20, 30, 40,
8)

9) 40, 30, 20, 4, 3, 1,
```

list示例程序

《程序设计实习》

```
lst2.push_back (100); lst2.push_back (200);
lst2.push_back (300); lst2.push_back (400);
list<A>::iterator p1, p2, p3;
p1 = find(lst1.begin(), lst1.end(), 3);
p2 = find(lst2.begin(), lst2.end(), 200);
p3 = find(lst2.begin(), lst2.end(), 400);
//将[p2, p3)插入p1之前, 并从lst2中删除[p2, p3)
lst1.splice(p1, lst2, p2, p3);
cout << "11) "; PrintList(lst1); cout << endl;
cout << "12) "; PrintList(lst2); cout << endl;
return 0;
}
```

```
lst1 40, 30, 20, 4, 3, 1,
lst2
```

```
11) 40, 30, 20, 4, 200, 300, 3, 1,
12) 100, 400,
```

- **vector**：强调通过**随机访问**进行快速访问
 - 插入/删除：非尾处 $O(n)$ 或结尾处 $O(1)$
- **deque**：类似**vector**，强调在**两端处**的**快速插入**和**删除**
 - 插入/删除：头尾处均为 $O(1)$
- **list**：强调元素的**快速插入**和**删除**，不支持**随机访问**迭代器
 - 插入/删除为 $O(1)$

- STL基本概念
- 容器概述
- 迭代器
- 算法简介
- STL中的大小相等比较
- 顺序容器：vector和双向队列deque
- 顺序容器：双向链表list
- 函数对象

- 一个类重载了()为成员函数 → 该类为函数对象类
- 这个类的对象 → 函数对象
 - 看上去像函数调用，实际上也执行了函数调用

```
#include <iostream>
using namespace std;

class Prod {
    int value;
public:
    Prod(int _v): value(_v){}
    int operator()(int other){
        return value * other;
    } //重载 ( ) 运算符
};
```

```
int main() {
    Prod p = Prod(2);
    cout << p(1) << endl;
    cout << p(2) << endl;
    return 0;
}
```

2
4

- 为了**STL算法可复用**，其中的子操作应该是参数化的
例如：sort的排序原则（顺序/ 逆序）
- 函数对象 就是用来描述这些**子操作的对象**

```
class CMyAverage {  
public:  
    double operator() (int a1, int a2, int a3) {  
        //重载 ( ) 运算符  
        return (double)(a1 + a2 + a3) / 3;  
    }  
}; //重载 ( ) 运算符时，参数可以是任意多个  
CMyAverage Average; //函数对象  
cout << Average(3, 2, 3);  
// Average.operator(3, 2, 3) 用起来看上去像函数调用，输出 2.66667
```

■ STL里有以下模板：

```
template<class InIt, class T, class Pred>
T accumulate(InIt first, InIt last, T val, Pred pr);
```

■ pr — 函数对象

对`[first, last)`中的每个迭代器 `I`,

执行 `val = pr(val, * I)`, 返回最终的`val`

■ Pr也可以是个函数名/函数指针

Dev C++中的Accumulate源代码：

《程序设计实习》

// 形式1：没有函数对象的版本，仅实现累加

```
template<typename _InputIterator, typename _Tp>
_Tp accumulate(_InputIterator __first, _InputIterator __last, _Tp __init) {
    for ( ; __first != __last; ++__first)
        __init = __init + *__first;
    return __init;
}
```

// 形式2：有函数对象的版本，根据函数对象定义的方式实现累加

```
template<typename _InputIterator, typename _Tp, typename _BinaryOperation>
_Tp accumulate(_InputIterator __first, _InputIterator __last,
               _Tp __init, _BinaryOperation __binary_op) {
    for ( ; __first != __last; ++__first)
        __init = __binary_op(__init, *__first);
    return __init;
}
```

调用accumulate时，和__binary_op对应的实参可以是 函数/函数对象

Accumulate函数示例

《程序设计实习》

```
#include <iostream>
#include <vector>
#include <numeric> //accumulate在此文件定义
#include <iterator>
using namespace std;
int SumSquares(int total, int value) {
    return total + value * value;
}
template<class T>
class SumPowers{
private:
    int power;
public:
    SumPowers(int p):power(p) { }
    const T operator( ) (const T & total, const T & value) {
        // 计算 value的power次方, 加到total上
        T v = value;
        for( int i = 0; i < power - 1; ++ i )
            v = v * value;
        return total + v;
    }
};
```


Accumulate函数示例

《程序设计实习》

```
int main() {  
    ostream_iterator<int> output(cout, " ");  
    const int SIZE = 10;  
    int a1[] = { 1,2,3,4,5,6,7,8,9,10 };  
    vector<int> v(a1, a1+SIZE);  
    cout << "1) "; copy(v.begin(), v.end(), output);  
    cout << endl;  
    int result = accumulate(v.begin(),v.end(),0,SumSquares);  
    cout << "2) 平方和: " << result << endl;  
    result = accumulate(v.begin(),v.end(),0,SumPowers<int>(3));  
    cout << "3) 立方和: " << result << endl;  
    result = accumulate(v.begin(),v.end(),0,SumPowers<int>(4));  
    cout << "4) 4次方和: " << result;  
    return 0;  
}
```

1) 1 2 3 4 5 6 7 8 9 10

2) 平方和: 385

3) 立方和: 3025

4) 4次方和: 25333

Accumulate函数示例

《程序设计实习》

```
result = accumulate(v.begin(), v.end(), 0, SumSquares);
```

■ 实例化出：

```
int accumulate(vector<int>::iterator first,
               vector<int>::iterator last,
               int init, int (* op)(int, int)) {
    for ( ; first != last; ++first)
        init = op(init, *first);
    return init;
}
```

Accumulate函数示例

《程序设计实习》

```
result = accumulate(v.begin(), v.end(), 0, SumPowers<int>(3))
```

■ 实例化出：

```
int accumulate(vector<int>::iterator first,
               vector<int>::iterator last,
               int init, SumPowers<int> op) {
    for ( ; first != last; ++first)
        init = op(init, *first);
    return init;
}
```

- STL的<functional>里还有以下函数对象类模板：
 - equal_to
 - greater
 - less
 - ...
- 这些模板可以用来生成函数对象

■ 根据函数对象参数个数，STL算法用到的主要有**三类基类**：

■ 没有参数的函数对象，相当于 "f()", 例如：

```
vector<int> V(10);  
generate(V.begin(), V.end(), rand);
```

■ 一个参数的函数对象，相当于 "f(x)"

STL中用unary_function定义了一元函数基类

```
template <class _Arg, class _Result>  
struct unary_function {  
    typedef _Arg argument_type;  
    typedef _Result result_type;  
};
```

■ 根据函数对象参数个数，STL算法用到的主要有**三类基类**：

■ 两个参数的函数对象，相当于 "f(x,y)"

STL中用binary_function定义了二元函数基类

```
template <class _Arg1, class _Arg2, class _Result>
struct binary_function {
    typedef _Arg1 first_argument_type;
    typedef _Arg2 second_argument_type;
    typedef _Result result_type;
};
```

注：这是 C++98 中的定义，C++11 中已更新

greater函数对象类模板

《程序设计实习》

```
template <class _Arg1, class _Arg2, class _Result>
struct binary_function {
    typedef _Arg1 first_argument_type;
    typedef _Arg2 second_argument_type;
    typedef _Result result_type;
};
```

```
template<class T>
struct greater : public binary_function<T, T, bool> {
    bool operator()(const T& x, const T& y) const {
        return x > y;
    }
};
```

■ list有两个sort函数

- 前面例子中看到的是**不带参数**的sort函数，
将list中的元素按 < 规定的比较方法**升序**排列
- list还有另一个sort函数：

```
void sort(op);    // 例子：void sort (greater<T> pr);
```

- 可以用op来比较大小，从而实现**降序**排序

greater的应用

《程序设计实习》

```
#include <list>
#include <iostream>
#include <iterator>
using namespace std;

int main(){
    const int SIZE = 5;
    int a[SIZE] = {5,21,14,2,3};
    list<int> lst(a, a+SIZE);
    lst.sort(greater<int>());
    //greater<int>()是个对象，本句进行降序排序
    ostream_iterator<int> output(cout, ",");
    copy(lst.begin(), lst.end(), output);
    cout << endl;
    return 0;
}
```

21,14,5,3,2,

greater的应用

《程序设计实习》

```
class MyLess {  
public:  
    bool operator()( const int & c1, const int & c2 ) {  
        return (c1 % 10) < (c2 % 10);  
    }  
};
```

```
int main(){  
    const int SIZE = 5;  
    int a[SIZE] = {5,21,14,2,3};  
    list<int> lst(a, a+SIZE);  
    lst.sort(MyLess());  
    ostream_iterator<int> output(cout, ",");  
    copy(lst.begin(), lst.end(), output);  
    cout << endl;  
    return 0;  
}
```

21,2,3,14,5,

例题MyMax

《程序设计实习》

```
#include <iostream>
#include <iterator>
using namespace std;
class MyLess {
public:
    bool operator() (int a1,int a2) {
        if( (a1 % 10) < (a2 % 10) )
            return true;
        else
            return false;
    }
};
bool MyCompare(int a1,int a2) {
    if( (a1 % 10) < (a2 % 10) )
        return false;
    else
        return true;
}
```

输出：

19

12

要求写出 MyMax

```
int main() {
    int a[] = {35,7,13,19,12};
    cout << MyMax(a,5,MyLess()) << endl;
    cout << MyMax(a,5,MyCompare) << endl;
    return 0;
}
```

例题MyMax

《程序设计实习》

```
#include <iostream>
#include <iterator>
using namespace std;
class MyLess {
public:
    bool operator() (int a1,int a2) {
        if( (a1 % 10) < (a2 % 10) )
            return true;
        else
            return false;
    }
};
bool MyCompare(int a1,int a2) {
    if( (a1 % 10) < (a2 % 10) )
        return false;
    else
        return true;
}
```

```
template <class T, class Pred>
T MyMax(T * p, int n, Pred myless) {
    T tmpmax = p[0];
    for( int i = 1; i < n; i ++ )
        if(myless(tmpmax, p[i]))
            tmpmax = p[i];
    return tmpmax;
};
```

```
int main() {
    int a[] = {35,7,13,19,12};
    cout << MyMax(a,5,MyLess()) << endl;
    cout << MyMax(a,5,MyCompare) << endl;
    return 0;
}
```


用sort进行排序

C++ STL

(Standard Template Library)
Ready-to-Use Components

《程序设计实习》

- 用自定义的排序规则，对任何类型T的数组排序

```
sort(数组名+n1, 数组名+n2, 排序规则结构名());
```

- 排序规则结构的定义方式

```
struct 结构名
{
    bool operator()( const T & a1, const T & a2) {
        //若a1应该在a2前面，则返回true。
        //否则返回false。
    }
};
```

用sort对结构数组进行排序

《程序设计实习》

```
struct Student {
    char name[20];
    int id;
    double gpa;
};
Student students [] = { {"Jack",112,3.4}, {"Mary",102,3.8}, {"Mary",117,3.9},
    {"Ala",333,3.5}, {"Zero",101,4.0}};
struct StudentRule1 { //按姓名从小到大排
    bool operator() (const Student & s1, const Student & s2) {
        if(stricmp(s1.name,s2.name) < 0) //大小写不敏感的strcmp
            return true;
        return false;
    }
};
struct StudentRule2 { //按id从小到大排
    bool operator() (const Student & s1, const Student & s2) {
        return s1.id < s2.id;
    }
};
struct StudentRule3 { //按gpa从高到低排
    bool operator() (const Student & s1, const Student & s2) {
        return s1.gpa > s2.gpa;
    }
};
```

用sort对结构数组进行排序

《程序设计实习》

```
void PrintStudents(Student s[],int size) {
    for(int i = 0; i < size; ++i)
        cout << "(" << s[i].name << ","
                << s[i].id <<"," << s[i].gpa << ") " ;
    cout << endl;
}
int main() {
    int n = sizeof(students) / sizeof(Student);
    sort(students,students+n,StudentRule1()); //按姓名从小到大排
    PrintStudents(students,n);
    sort(students,students+n,StudentRule2()); //按id从小到大排
    PrintStudents(students,n);
    sort(students,students+n,StudentRule3()); //按gpa从高到低排
    PrintStudents(students,n);
    return 0;
}
```

(Ala,333,3.5) (Jack,112,3.4) (Mary,102,3.8) (Mary,117,3.9) (Zero,101,4)
(Zero,101,4) (Mary,102,3.8) (Jack,112,3.4) (Mary,117,3.9) (Ala,333,3.5)
(Zero,101,4) (Mary,117,3.9) (Mary,102,3.8) (Ala,333,3.5) (Jack,112,3.4)